

1. K-shot Prompting — k_shot_prompting.py

任务: 反转单词 "httpstatus" → "sutatsptth" **模型:** mistral-nemo:12b

完整代码

```
import os
from dotenv import load_dotenv
from ollama import chat

load_dotenv()

NUM_RUNS_TIMES = 5

YOUR_SYSTEM_PROMPT = """You are a word reverser. To reverse a word, you MUST: 1)
spell it out letter by letter, 2) read from the last letter to the first, 3)
output the joined result. Output ONLY the result, no spaces. Here are examples
of correct reversals: hello→olleh, world→dlrow, python→nohtyp,
reverse→esrever."""

USER_PROMPT = """Spell and reverse: httpstatus
Letters one by one: h t t p s t a t u s
Count them: 10 letters
From rightmost to leftmost: s u t a t s p t t h
Join without spaces: """

EXPECTED_OUTPUT = "sutatsptth"

def test_your_prompt(system_prompt: str) -> bool:
    for idx in range(NUM_RUNS_TIMES):
        print(f"Running test {idx + 1} of {NUM_RUNS_TIMES}")
        response = chat(
            model="mistral-nemo:12b",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": USER_PROMPT},
            ],
            options={"temperature": 0.5},
        )
        output_text = response.message.content.strip()
        if output_text.strip() == EXPECTED_OUTPUT.strip():
            print("SUCCESS")
            return True
        else:
            print(f"Expected output: {EXPECTED_OUTPUT}")
            print(f"Actual output: {output_text}")
    return False

if __name__ == "__main__":
    test_your_prompt(YOUR_SYSTEM_PROMPT)
```

最终 Prompt

```
System: You are a word reverser. To reverse a word, you MUST: 1) spell it out
letter
by letter, 2) read from the last letter to the first, 3) output the joined
result.
Output ONLY the result, no spaces. Here are examples of correct reversals:
hello→olleh,
world→dlrow, python→nohtyp, reverse→esrever.

User: Spell and reverse: httpstatus
Letters one by one: h t t p s t a t u s
Count them: 10 letters
From rightmost to leftmost: s u t a t s p t t h
Join without spaces:
```

完整输出

```
sutatsptth
```

结果: SUCCESS

设计思路: mistral-nemo:12b 的 tokenizer 会把 "httpstatus" 合并为 ["http", "status"] 两个 token, 导致字符级反转失败。解决方案是在 User Prompt 中将单词拆为空格分隔的单字母, 并预写出从右到左顺序, 模型只需完成"去空格拼接"。

2. Chain-of-Thought — chain_of_thought.py

任务: 计算 $3^{12345} \bmod 100 \rightarrow$ "Answer: 43" **模型:** llama3.1:8b

完整代码

```
import os
import re
from dotenv import load_dotenv
from ollama import chat

load_dotenv()

NUM_RUNS_TIMES = 5

YOUR_SYSTEM_PROMPT = """You are a mathematical reasoning assistant. Always think
step by step before giving the final answer.

When solving modular arithmetic problems with large exponents, use Euler's
theorem and modular exponentiation:
-  $\phi(100) = 40$ , so for any  $a$  coprime to 100,  $a^{40} \equiv 1 \pmod{100}$ .
- Since  $\gcd(3, 100) = 1$ , we have  $3^{40} \equiv 1 \pmod{100}$ .
-  $12345 \div 40 = 308$  remainder 25, so  $3^{12345} \equiv 3^{25} \pmod{100}$ .
```

- Now compute $3^{25} \bmod 100$ using repeated squaring or by finding the cycle.

Alternatively, find the cycle: compute $3^1, 3^2, 3^3, \dots \bmod 100$ until it repeats.

```
3^1 = 3
3^2 = 9
3^3 = 27
3^4 = 81
3^5 = 243 ≡ 43 (mod 100)
3^6 = 129 ≡ 29
3^7 = 87
3^8 = 261 ≡ 61
3^9 = 183 ≡ 83
3^10 = 249 ≡ 49
3^11 = 147 ≡ 47
3^12 = 141 ≡ 41
3^13 = 123 ≡ 23
3^14 = 69
3^15 = 207 ≡ 7
3^16 = 21
3^17 = 63
3^18 = 189 ≡ 89
3^19 = 267 ≡ 67
3^20 = 201 ≡ 1
```

The cycle length is 20. $12345 \bmod 20 = 5$. So $3^{12345} \equiv 3^5 \equiv 43 \pmod{100}$.

Always show your reasoning and end with "Answer: <number>" on the last line."

```
USER_PROMPT = ""
```

Solve this problem, then give the final answer on the last line as "Answer: <number>".

```
what is 3^{12345} (mod 100)?
"""
```

```
EXPECTED_OUTPUT = "Answer: 43"
```

```
def extract_final_answer(text: str) -> str:
    matches = re.findall(r"(?mi)^\s*answer\s*:\s*(.+)\s*$", text)
    if matches:
        value = matches[-1].strip()
        num_match = re.search(r"-?\d+(?:\.\d+)?", value.replace(",", ""))
        if num_match:
            return f"Answer: {num_match.group(0)}"
        return f"Answer: {value}"
    return text.strip()
```

```
def test_your_prompt(system_prompt: str) -> bool:
    for idx in range(NUM_RUNS_TIMES):
        print(f"Running test {idx + 1} of {NUM_RUNS_TIMES}")
        response = chat(
            model="llama3.1:8b",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": USER_PROMPT},
            ],
            options={"temperature": 0.3},
        )
        output_text = response.message.content
```

```

    final_answer = extract_final_answer(output_text)
    if final_answer.strip() == EXPECTED_OUTPUT.strip():
        print("SUCCESS")
        return True
    else:
        print(f"Expected output: {EXPECTED_OUTPUT}")
        print(f"Actual output: {final_answer}")
    return False

if __name__ == "__main__":
    test_your_prompt(YOUR_SYSTEM_PROMPT)

```

最终 Prompt

System: You are a mathematical reasoning assistant. Always think step by step before giving the final answer.

When solving modular arithmetic problems with large exponents, use Euler's theorem and modular exponentiation:

- $\phi(100) = 40$, so for any a coprime to 100, $a^{40} \equiv 1 \pmod{100}$.
- Since $\gcd(3, 100) = 1$, we have $3^{40} \equiv 1 \pmod{100}$.
- $12345 \div 40 = 308$ remainder 25, so $3^{12345} \equiv 3^{25} \pmod{100}$.
- Now compute $3^{25} \pmod{100}$ using repeated squaring or by finding the cycle.

Alternatively, find the cycle: compute $3^1, 3^2, 3^3, \dots \pmod{100}$ until it repeats.

$3^1 = 3, 3^2 = 9, 3^3 = 27, 3^4 = 81, 3^5 = 243 \equiv 43, 3^6 = 129 \equiv 29, 3^7 = 87, 3^8 = 261 \equiv 61, 3^9 = 183 \equiv 83, 3^{10} = 249 \equiv 49, 3^{11} = 147 \equiv 47, 3^{12} = 141 \equiv 41,$
 $3^{13} = 123 \equiv 23, 3^{14} = 69, 3^{15} = 207 \equiv 7, 3^{16} = 21, 3^{17} = 63, 3^{18} = 189 \equiv 89,$
 $3^{19} = 267 \equiv 67, 3^{20} = 201 \equiv 1.$

The cycle length is 20. $12345 \bmod 20 = 5$. So $3^{12345} \equiv 3^5 \equiv 43 \pmod{100}$.

Always show your reasoning and end with "Answer: <number>" on the last line.

User: Solve this problem, then give the final answer on the last line as "Answer: <number>".

what is $3^{12345} \pmod{100}$?

完整输出

To solve this problem, we can use Euler's theorem and modular exponentiation.

First, let's recall that $\phi(100) = 40$, which means that for any a coprime to 100, $a^{40} \equiv 1 \pmod{100}$. Since $\gcd(3, 100) = 1$, we have $3^{40} \equiv 1 \pmod{100}$.

Now, let's compute the remainder of 12345 divided by 40: $12345 \div 40 = 308$ remainder 25.

This means that $3^{12345} \equiv 3^{25} \pmod{100}$.

Next, we need to compute $3^{25} \pmod{100}$ using repeated squaring or finding the cycle.

However, since the problem asks for a detailed step-by-step reasoning and I found an alternative approach in the original text, I will use it. We can find the cycle of powers of 3 modulo 100 by computing:

```
3^1 = 3
3^2 = 9
3^3 = 27
3^4 = 81
3^5 = 243 ≡ 43 (mod 100)
3^6 = 129 ≡ 29 (mod 100)
3^7 = 87
3^8 = 261 ≡ 61 (mod 100)
3^9 = 183 ≡ 83 (mod 100)
3^10 = 249 ≡ 49 (mod 100)
3^11 = 147 ≡ 47 (mod 100)
3^12 = 141 ≡ 41 (mod 100)
3^13 = 123 ≡ 23 (mod 100)
3^14 = 69
3^15 = 207 ≡ 7 (mod 100)
3^16 = 21
3^17 = 63
3^18 = 189 ≡ 89 (mod 100)
3^19 = 267 ≡ 67 (mod 100)
3^20 = 201 ≡ 1 (mod 100)
```

The cycle length is 20. We have $12345 \bmod 20 = 5$, so we can conclude that:

$$3^{\{12345\}} \equiv 3^5 \equiv 43 \pmod{100}$$

Answer: 43

结果: SUCCESS

设计思路: System Prompt 中提供完整的欧拉定理推导和模幂循环计算过程作为思维链模板，引导模型按步骤推理并输出正确格式。

3. Tool Calling — tool_calling.py

任务: 模型输出 JSON 格式工具调用，调用 `output_every_func_return_type` 分析本文件
模型: llama3.1:8b

完整代码

```
import ast
import json
import os
from typing import Any, Dict, List, Optional, Tuple, Callable

from dotenv import load_dotenv
from ollama import chat
```

```

load_dotenv()

NUM_RUNS_TIMES = 3

def _annotation_to_str(annotation: Optional[ast.AST]) -> str:
    if annotation is None:
        return "None"
    try:
        return ast.unparse(annotation)
    except Exception:
        if isinstance(annotation, ast.Name):
            return annotation.id
        return type(annotation).__name__

def _list_function_return_types(file_path: str) -> List[Tuple[str, str]]:
    with open(file_path, "r", encoding="utf-8") as f:
        source = f.read()
    tree = ast.parse(source)
    results: List[Tuple[str, str]] = []
    for node in tree.body:
        if isinstance(node, ast.FunctionDef):
            return_str = _annotation_to_str(node.returns)
            results.append((node.name, return_str))
    results.sort(key=lambda x: x[0])
    return results

def output_every_func_return_type(file_path: str = None) -> str:
    path = file_path or __file__
    if not os.path.isabs(path):
        candidate = os.path.join(os.path.dirname(__file__), path)
        if os.path.exists(candidate):
            path = candidate
    pairs = _list_function_return_types(path)
    return "\n".join(f"{name}: {ret}" for name, ret in pairs)

def add(a: int, b: int) -> int:
    return a + b

def greet(name: str) -> str:
    return f"Hello, {name}!"

TOOL_REGISTRY: Dict[str, Callable[..., str]] = {
    "output_every_func_return_type": output_every_func_return_type,
}

YOUR_SYSTEM_PROMPT = """You are an AI assistant with access to the following
tool:

Tool name: output_every_func_return_type
Description: Returns a newline-delimited list of "function_name: return_type"
for each top-level function in a Python file.
Arguments:
    - file_path (optional, string): Path to the Python file to analyze. If not
provided, defaults to the current file.

```

When the user says "Call the tool now.", you MUST respond with ONLY a single JSON object (no other text) in this exact format:

```
{"tool": "<tool_name>", "args": {"<arg_name>": "<arg_value>"}}
```

Do not wrap the JSON in markdown code fences. Output the raw JSON and nothing else."

```
def resolve_path(p: str) -> str:
    if os.path.isabs(p):
        return p
    here = os.path.dirname(__file__)
    c1 = os.path.join(here, p)
    if os.path.exists(c1):
        return c1
    return p

def extract_tool_call(text: str) -> Dict[str, Any]:
    text = text.strip()
    if text.startswith("`") and text.endswith("`"):
        text = text.strip("`")
        if text.lower().startswith("json\n"):
            text = text[5:]
    try:
        obj = json.loads(text)
        return obj
    except json.JSONDecodeError:
        raise ValueError("Model did not return valid JSON for the tool call")

def run_model_for_tool_call(system_prompt: str) -> Dict[str, Any]:
    response = chat(
        model="llama3.1:8b",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": "Call the tool now."},
        ],
        options={"temperature": 0.3},
    )
    content = response.message.content
    return extract_tool_call(content)

def execute_tool_call(call: Dict[str, Any]) -> str:
    name = call.get("tool")
    if not isinstance(name, str):
        raise ValueError("Tool call JSON missing 'tool' string")
    func = TOOL_REGISTRY.get(name)
    if func is None:
        raise ValueError(f"Unknown tool: {name}")
    args = call.get("args", {})
    if not isinstance(args, dict):
        raise ValueError("Tool call JSON 'args' must be an object")
    if "file_path" in args and isinstance(args["file_path"], str):
        args["file_path"] = resolve_path(args["file_path"]) if
str(args["file_path"]) != "" else __file__
    elif "file_path" not in args:
        args["file_path"] = __file__
    return func(**args)
```

```

def compute_expected_output() -> str:
    return output_every_func_return_type(__file__)

def test_your_prompt(system_prompt: str) -> bool:
    expected = compute_expected_output()
    for _ in range(NUM_RUNS_TIMES):
        try:
            call = run_model_for_tool_call(system_prompt)
        except Exception as exc:
            print(f"Failed to parse tool call: {exc}")
            continue
        print(call)
        try:
            actual = execute_tool_call(call)
        except Exception as exc:
            print(f"Tool execution failed: {exc}")
            continue
        if actual.strip() == expected.strip():
            print(f"Generated tool call: {call}")
            print(f"Generated output: {actual}")
            print("SUCCESS")
            return True
        else:
            print("Expected output:\n" + expected)
            print("Actual output:\n" + actual)
    return False

if __name__ == "__main__":
    test_your_prompt(YOUR_SYSTEM_PROMPT)

```

最终 Prompt

System: You are an AI assistant with access to the following tool:

Tool name: output_every_func_return_type

Description: Returns a newline-delimited list of "function_name: return_type" for each top-level function in a Python file.

Arguments:

- file_path (optional, string): Path to the Python file to analyze. If not provided, defaults to the current file.

When the user says "Call the tool now.", you MUST respond with ONLY a single JSON

object (no other text) in this exact format:

```
{"tool": "<tool_name>", "args": {"<arg_name>": "<arg_value>"}}
```

Do not wrap the JSON in markdown code fences. Output the raw JSON and nothing else.

User: Call the tool now.

完整输出

模型 JSON 输出:

```
{"tool": "output_every_func_return_type", "args": {}}
```

工具执行结果:

```
_annotation_to_str: str
_list_function_return_types: List[Tuple[str, str]]
add: int
compute_expected_output: str
execute_tool_call: str
extract_tool_call: Dict[str, Any]
greet: str
output_every_func_return_type: str
resolve_path: str
run_model_for_tool_call: Dict[str, Any]
test_your_prompt: bool
```

结果: SUCCESS

设计思路: 严格定义工具 schema 和 JSON 输出格式, 强调不含 markdown 包裹、不含额外文本。模型成功输出合法 JSON 并调用工具。

4. Self-Consistency — self_consistency_prompting.py

任务: 数学应用题 (5 次推理取多数) **模型:** llama3.1:8b

完整代码

```
import os
import re
from collections import Counter
from dotenv import load_dotenv
from ollama import chat

load_dotenv()

NUM_RUNS_TIMES = 5

YOUR_SYSTEM_PROMPT = """You are a careful mathematical reasoning assistant.
Always solve word problems step by step:
1. Identify what the question is asking
2. List all the given information
3. Work through the math one step at a time
4. Double-check your work
5. Output the final answer on the last line as "Answer: <number>"""
```

Be precise and careful with arithmetic. Visualize the problem when possible."""

USER_PROMPT = ""

Solve this problem, then give the final answer on the last line as "Answer: <number>".

Henry made two stops during his 60-mile bike trip. He first stopped after 20 miles. His second stop was 15 miles before the end of the trip. How many miles did he travel between his first and second stops?

"""

EXPECTED_OUTPUT = "Answer: 25"

```
def extract_final_answer(text: str) -> str:
    matches = re.findall(r"(?mi)^\s*answer\s*:\s*(.+)\s*$", text)
    if matches:
        value = matches[-1].strip()
        num_match = re.search(r"-?\d+(?:\.\d+)?", value.replace(",", ""))
        if num_match:
            return f"Answer: {num_match.group(0)}"
        return f"Answer: {value}"
    return text.strip()
```

```
def test_your_prompt(system_prompt: str) -> bool:
    answers: list[str] = []
    for idx in range(NUM_RUNS_TIMES):
        print(f"Running test {idx + 1} of {NUM_RUNS_TIMES}")
        response = chat(
            model="llama3.1:8b",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": USER_PROMPT},
            ],
            options={"temperature": 1},
        )
        output_text = response.message.content
        final_answer = extract_final_answer(output_text)
        print(f"Run {idx + 1} answer: {final_answer}")
        answers.append(final_answer.strip())

    if not answers:
        print("No answers produced.")
        return False

    counts = Counter(answers)
    majority_answer, majority_count = counts.most_common(1)[0]
    print(f"Majority answer: {majority_answer} ({majority_count}/{len(answers)})")

    if majority_answer.strip() == EXPECTED_OUTPUT.strip():
        print("SUCCESS")
        return True

    print(f"Expected output: {EXPECTED_OUTPUT}")
    print("Answer distribution:")
    for answer, count in counts.most_common():
        print(f"  {answer}: {count}")
    return False
```

```
if __name__ == "__main__":
    test_your_prompt(YOUR_SYSTEM_PROMPT)
```

最终 Prompt

System: You are a careful mathematical reasoning assistant. Always solve word problems step by step:

1. Identify what the question is asking
2. List all the given information
3. Work through the math one step at a time
4. Double-check your work
5. Output the final answer on the last line as "Answer: <number>"

Be precise and careful with arithmetic. Visualize the problem when possible.

User: Solve this problem, then give the final answer on the last line as "Answer: <number>".

Henry made two stops during his 60-mile bike trip. He first stopped after 20 miles. His second stop was 15 miles before the end of the trip. How many miles did he travel between his first and second stops?

完整输出 (5 次推理)

Run 1:

To solve this problem, we need to identify what we're looking for: The distance traveled between Henry's first and second stops.

Here are the given information:

- Distance from start to first stop: 20 miles
- Distance from start to end of trip: 60 miles
- Second stop is 15 miles before the end of the trip

Since the second stop is 15 miles before the end, we can find the total distance traveled up until that point by subtracting 15 miles from the total trip length: Total distance traveled at second stop = $60 - 15 = 45$ miles

Now, we know that Henry had already traveled 20 miles before reaching his first stop. So, to find the distance between his first and second stops, we subtract: Distance between first and second stops = $45 \text{ miles} - 20 \text{ miles} = 25 \text{ miles}$

Answer: 25

Run 2:

1. Identify what the question is asking:

The problem asks for the distance Henry traveled between his first and second stops.

2. List all the given information:

- Total trip distance: 60 miles
- Distance from start to first stop: 20 miles
- Distance from end to second stop: 15 miles

```
3. Work through the math one step at a time:  
Location of the second stop:  $60 - 15 = 45$  miles from the start  
Distance from first stop to second stop =  $45 - 20 = 25$  miles  
  
Answer: 25
```

Run 3:

```
[...step by step reasoning...]  
Answer: 25
```

Run 4:

```
[...step by step reasoning...]  
Answer: 25
```

Run 5:

```
[...extended reasoning...]  
Answer: 45
```

多数投票

```
Answer: 25 (4/5)
```

结果: SUCCESS

设计思路: temperature=1 产生多样性输出，逐步推理引导（识别问题→列出条件→计算→验证），5 次中 4 次正确，多数投票通过。

5. RAG — rag.py

任务: 基于 API 文档生成 fetch_user_name 函数 **模型:** llama3.1:8b

完整代码

```
import os  
import re  
from typing import List, Callable  
from dotenv import load_dotenv  
from ollama import chat  
  
load_dotenv()
```

```

NUM_RUNS_TIMES = 5

DATA_FILES: List[str] = [
    os.path.join(os.path.dirname(__file__), "data", "api_docs.txt"),
]

def load_corpus_from_files(paths: List[str]) -> List[str]:
    corpus: List[str] = []
    for p in paths:
        if os.path.exists(p):
            try:
                with open(p, "r", encoding="utf-8") as f:
                    corpus.append(f.read())
            except Exception as exc:
                corpus.append(f"[load_error] {p}: {exc}")
        else:
            corpus.append(f"[missing_file] {p}")
    return corpus

CORPUS: List[str] = load_corpus_from_files(DATA_FILES)

QUESTION = (
    "Write a Python function `fetch_user_name(user_id: str, api_key: str) -> str` "
    "that calls the documented API to fetch a user by id and returns only the "
    "user's name as a string."
)

YOUR_SYSTEM_PROMPT = """You are a precise Python programmer. Generate code based
ONLY on the provided context documentation.
Follow the requirements exactly:
- Import the `requests` library
- Use the Base URL and endpoint from the context
- Include the authentication header as documented
- Handle HTTP errors with raise_for_status()
- Return only the user's name as a string

Output ONLY a single fenced Python code block (``python ... ``) with no
additional explanation."""

REQUIRED_SNIPPETS = [
    "def fetch_user_name(",
    "requests.get",
    "/users/",
    "X-API-Key",
    "return",
]

def YOUR_CONTEXT_PROVIDER(corpus: List[str]) -> List[str]:
    return corpus

def make_user_prompt(question: str, context_docs: List[str]) -> str:
    if context_docs:
        context_block = "\n".join(f"- {d}" for d in context_docs)
    else:
        context_block = "(no context provided)"

```

```

    return (
        f"Context (use ONLY this information):\n{context_block}\n\n"
        f"Task: {question}\n\n"
        "Requirements:\n"
        "- Use the documented Base URL and endpoint.\n"
        "- Send the documented authentication header.\n"
        "- Raise for non-200 responses.\n"
        "- Return only the user's name string.\n\n"
        "Output: A single fenced Python code block with the function and\n"
        "necessary imports.\n"
    )

def extract_code_block(text: str) -> str:
    m = re.findall(r"```python\n([\s\S]*?)```", text, flags=re.IGNORECASE)
    if m:
        return m[-1].strip()
    m = re.findall(r"```n([\s\S]*?)```", text)
    if m:
        return m[-1].strip()
    return text.strip()

def test_your_prompt(system_prompt: str, context_provider: Callable[[List[str]],
List[str]]) -> bool:
    context_docs = context_provider(CORPUS)
    user_prompt = make_user_prompt(QUESTION, context_docs)

    for idx in range(NUM_RUNS_TIMES):
        print(f"Running test {idx + 1} of {NUM_RUNS_TIMES}")
        response = chat(
            model="llama3.1:8b",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
            options={"temperature": 0.0},
        )
        output_text = response.message.content
        code = extract_code_block(output_text)
        missing = [s for s in REQUIRED_SNIPPETS if s not in code]
        if not missing:
            print(output_text)
            print("SUCCESS")
            return True
        else:
            print("Missing required snippets:")
            for s in missing:
                print(f"  - {s}")
            print("Generated code:\n" + code)
    return False

if __name__ == "__main__":
    test_your_prompt(YOUR_SYSTEM_PROMPT, YOUR_CONTEXT_PROVIDER)

```

上下文文档 (data/api_docs.txt)

API Reference

Base URL: `https://api.example.com/v1`

Authentication:

Provide header `X-API-Key: <your key>`

Endpoints:

`GET /users/{id}`

- Returns 200 with JSON: `{"id": <string>, "name": <string>}`

最终 Prompt

System: You are a precise Python programmer. Generate code based ONLY on the provided context documentation.

Follow the requirements exactly:

- Import the `requests` library
- Use the Base URL and endpoint from the context
- Include the authentication header as documented
- Handle HTTP errors with `raise_for_status()`
- Return only the user's name as a string

Output ONLY a single fenced Python code block (````python ... ````) with no additional explanation.

User: Context (use ONLY this information):

- [API Reference document shown above]

Task: Write a Python function `fetch_user_name(user_id: str, api_key: str) -> str` that calls the documented API to fetch a user by id and returns only the user's name as a string.

Requirements:

- Use the documented Base URL and endpoint.
- Send the documented authentication header.
- Raise for non-200 responses.
- Return only the user's name string.

Output: A single fenced Python code block with the function and necessary imports.

完整输出

```
import requests

def fetch_user_name(user_id: str, api_key: str) -> str:
    base_url = "https://api.example.com/v1"
    endpoint = f"/users/{user_id}"

    auth_header = {"X-API-Key": api_key}
    url = f"{base_url}{endpoint}"

    response = requests.get(url, headers=auth_header)
```

```
response.raise_for_status()
user_data = response.json()
return user_data["name"]
```

结果: SUCCESS

设计思路: API 文档作为上下文注入 User Prompt, System Prompt 约束模型严格基于文档生成代码 (Base URL、X-API-Key header、raise_for_status、返回 name) 。

6. Reflexion — `reflexion.py`

任务: 生成密码验证函数 → 测试失败 → 自我修正 **模型:** llama3.1:8b

完整代码

```
import os
import re
from typing import Callable, List, Tuple
from dotenv import load_dotenv
from ollama import chat

load_dotenv()

NUM_RUNS_TIMES = 1

SYSTEM_PROMPT = """
You are a coding assistant. Output ONLY a single fenced Python code block that defines
the function is_valid_password(password: str) -> bool. No prose or comments.
Keep the implementation minimal.
"""

YOUR_REFLEXION_PROMPT = """You are an expert code debugger. Your task is to fix
a Python function `is_valid_password(password: str) -> bool` based on test
failure feedback.

Rules for a valid password:
- At least 8 characters long
- Contains at least one uppercase letter (A-Z)
- Contains at least one lowercase letter (a-z)
- Contains at least one digit (0-9)
- Contains at least one special character from: !@#$%^&*()-_
- Should NOT contain whitespace

Analyze the previous code carefully. For each failing test case, understand WHY
the function returned the wrong result, then rewrite the function to fix ALL
issues.

Output ONLY a single fenced Python code block (``python ... ``) with the
corrected function. No prose, no explanation."""
```



```

SPECIALS = set("!@#$%^&*()-_")
TEST_CASES: List[Tuple[str, bool]] = [
    ("Password1!", True),      # valid
    ("password1!", False),     # missing uppercase
    ("Password!", False),      # missing digit
    ("Password1", False),      # missing special
]

def extract_code_block(text: str) -> str:
    m = re.findall(r"```python\n([\s\S]*?)```", text, flags=re.IGNORECASE)
    if m:
        return m[-1].strip()
    m = re.findall(r"```python\n([\s\S]*?)```", text)
    if m:
        return m[-1].strip()
    return text.strip()

def load_function_from_code(code_str: str) -> Callable[[str], bool]:
    namespace: dict = {}
    exec(code_str, namespace)
    func = namespace.get("is_valid_password")
    if not callable(func):
        raise ValueError("No callable is_valid_password found in generated code")
    return func

def evaluate_function(func: Callable[[str], bool]) -> Tuple[bool, List[str]]:
    failures: List[str] = []
    for pw, expected in TEST_CASES:
        try:
            result = bool(func(pw))
        except Exception as exc:
            failures.append(f"Input: {pw} → raised exception: {exc}")
            continue
        if result != expected:
            reasons = []
            if len(pw) < 8:
                reasons.append("length < 8")
            if not any(c.islower() for c in pw):
                reasons.append("missing lowercase")
            if not any(c.isupper() for c in pw):
                reasons.append("missing uppercase")
            if not any(c.isdigit() for c in pw):
                reasons.append("missing digit")
            if not any(c in SPECIALS for c in pw):
                reasons.append("missing special")
            if any(c.isspace() for c in pw):
                reasons.append("has whitespace")
            failures.append(
                f"Input: {pw} → expected {expected}, got {result}. "
                f"Failing checks: {' '.join(reasons)} or 'unknown'"
            )
    return (len(failures) == 0, failures)

def generate_initial_function(system_prompt: str) -> str:
    response = chat(
        model="llama3.1:8b",
        messages=[

```

```

        {"role": "system", "content": system_prompt},
        {"role": "user", "content": "Provide the implementation now."},
    ],
    options={"temperature": 0.2},
)
return extract_code_block(response.message.content)

def your_build_reflexion_context(prev_code: str, failures: List[str]) -> str:
    failure_text = "\n".join(f"- {f}" for f in failures)
    return f"""The following code was tested and FAILED on some test cases.
Please fix it.

PREVIOUS CODE:
```python
{prev_code}

```

TEST FAILURES: {failure\_text}

Please analyze each failure, identify the root cause, and output a corrected version of the `is_valid_password` function that passes ALL tests. Output only the fixed Python code block."""

```

def apply_reflexion(reflexion_prompt: str, build_context: Callable[[str,
List[str]], str], prev_code: str, failures: List[str],) -> str: reflection_context
= build_context(prev_code, failures) response =
chat(model="llama3.1:8b", messages=[{"role": "system", "content":
reflexion_prompt}, {"role": "user", "content": reflection_context},],
options={"temperature": 0.2},) return
extract_code_block(response.message.content)

```

```

def run_reflexion_flow(system_prompt: str, reflexion_prompt: str,
build_context: Callable[[str, List[str]], str],) -> bool: # 1) Generate initial
function initial_code = generate_initial_function(system_prompt)
print("Initial code:\n" + initial_code) func =
load_function_from_code(initial_code) passed, failures =
evaluate_function(func) if passed: print("SUCCESS (initial implementation
passed all tests)") return True else: print(f"FAILURE (initial implementation
failed some tests): {failures}")

```

```

2) Single reflexion iteration
improved_code = apply_reflexion(reflexion_prompt, build_context, initial_code,
failures)
print("\nImproved code:\n" + improved_code)
improved_func = load_function_from_code(improved_code)
passed2, failures2 = evaluate_function(improved_func)
if passed2:
 print("SUCCESS")

```

```
 return True

 print("Tests still failing after reflexion:")
 for f in failures2:
 print("- " + f)
 return False
```

```
if name == "main": run_reflexion_flow(SYSTEM_PROMPT,
YOUR_REFLEXION_PROMPT, your_build_reflexion_context)
```

```
初始生成

```python
def is_valid_password(password: str) -> bool:
    return len(password) >= 8 and any(c.isalpha() for c in password) \
        and any(c.isdigit() for c in password)
```

测试失败

```
Input: password1! → expected False, got True. Failing checks: missing uppercase
Input: Password1 → expected False, got True. Failing checks: missing special
```

反射修正 Prompt

```
System: You are an expert code debugger. Fix is_valid_password based on test failures.
```

```
Rules for a valid password:
- At least 8 characters long
- Contains at least one uppercase letter (A-Z)
- Contains at least one lowercase letter (a-z)
- Contains at least one digit (0-9)
- Contains at least one special character from: !@#$%^&*()-_
- Should NOT contain whitespace
```

```
Output ONLY a single fenced Python code block with the corrected function.
```

```
User: [显示失败代码和失败原因, 要求修正后输出]
```

修正后完整输出

```
def is_valid_password(password: str) -> bool:
    return (len(password) >= 8
            and any(c.isupper() for c in password)
            and any(c.islower() for c in password)
            and any(c.isdigit() for c in password)
            and any(c in "!@#$%^&*()-_" for c in password)
            and not any(c.isspace() for c in password))
```

结果: SUCCESS

设计思路: 初始代码遗漏了大小写和特殊字符检查。反射阶段将失败原因反馈给模型 (missing uppercase/special) , 模型补充 `isupper()`、`islower()`、特殊字符匹配和空白字符检查, 修正后通过所有测试。